Database Technologies (DBT) - Day 22 and Day 23

Normalization, MySQL Architecture and Storage Engines, Codd's Rules, Temporary Tables, NoSQL and MongoDB Basics

Lecture notes - Notes for: Sumeet Sharma

#	Topic covered	Day
1	Project guidance and why normalization must be seen from the project's point of view	22
2	Getting ready for normalization (unnormalized form) and the three anomalies	22
3	1NF, 2NF, 3NF and BCNF with one running example; practical statistics	22
4	Effect of normalization on queries and denormalization	22
5	MySQL architecture, storage engines (InnoDB vs MyISAM and others)	22
6	Temporary tables vs views	22
7	Codd's rules (the foundation rule and the 12 rules)	22
8	USING and NATURAL JOIN	22
9	Introduction to NoSQL: history, features, BASE, vertical vs horizontal scaling	22
10	CAP theorem, replica sets and partitions	23
11	When to use NoSQL and when not to; hybrid use; migration	23
12	Types of NoSQL databases (key-value, column, graph, document)	23
13	MongoDB: JSON, BSON, installation, architecture, shell	23
14	MongoDB CRUD: insert, find, projection, operators, skip/limit/sort, delete, update	23
15	ObjectId, validators, indexes, aggregation, $lookup, transactions, GridFS and Q&A	23
16	Revision: summary, definitions, differences, mistakes, interview Q&A, exercises, checklist	-

How to read these notes
Blue boxes hold the instructor's explanation in clear written English. Yellow boxes are additional explanations added by me; they were **not** said in the lecture. Red boxes list common mistakes. The two recordings are noisy (speech to text with many errors), so wherever I reconstructed an example, number or command I say so. The first recording (Day 22) runs from normalization to the start of NoSQL; the second (Day 23, an evening session) continues with NoSQL and MongoDB. Class housekeeping, breaks and technical help for individual students are left out.

PART I - DAY 22

1. Project Guidance and the Project View of Normalization

Instructor's explanation
The normalization lecture should not be limited to normalization itself; it should make you **think from your project's perspective**. Students who treat their project seriously go through requirement analysis and design, and then they understand normalization well. Others find it boring or difficult, or ask hypothetical, unnecessary questions. Before you can complete the normalization of your project you must first complete the **UI design**, and for that you must first understand which project you want to do and what functionality you want to provide.

A few pieces of practical advice from the lecture:

- The first week of any module feels relaxed; each module then gets tougher. The next six to seven days are a good window to start the project and set a target. Once the Java module becomes complicated you will not be able to concentrate on both the project and Java, and you will forget the process of normalization.
- The project will be done in a group; ask your questions on your Zoom channels so that course coordinators take the initiative (partner selection, mechanisms).
- Do not spend months deciding which project to do and then build something one day before the presentation. The aim of a C-DAC type project is **implementation**, not a fancy idea. Companies do not expect research papers; they hire you for your implementation skills.
- Did you go through all phases: design, implementation, testing, deployment? Companies then ask implementation questions: how did you design the tables, why did you split a table, what is the logic? A reply like 'I felt it should be separate' is risky. They may ask you to write the ten lines of code of a feature on paper or Notepad.
- Do not waste time hunting for an idea. Take any idea, visit existing websites in that domain, create accounts and understand them, **define the scope and gather requirements**, then do UI design and database design as fast as you can. Create a Git repository once your project partner is final and start documenting there.
- Get your scope and team size approved by your project mentors; they decide how much functionality is valid for your team size. Your contribution in the team project is asked in interviews: saying you did everything alone suggests either lying or being a poor team player.
- A project is typically 80 to 100 hours of work (about 15 full days). Start early; do not wait for the course to finish.

Additional note (not from the lecture)
The instructor said the idea is not marked and that implementation is. The scope approval step (mentors confirming that the amount of functionality matches your team size) is how his institute handles it; follow your own course coordinators' rules.

2. Why Normalize? Getting Ready and the Three Anomalies

2.1 Getting ready for normalization (the unnormalized form, UNF)

This part was done in the previous session and is recalled here. The steps on the slide, in the instructor's words:

1. For a given **transaction**, make a list of all fields (one transaction = one customer placing one order, with all the data related to that order).
2. Get a general description of all fields; for all practical purposes we can have a **single table with all columns**; assign data types and widths to all columns.

3. **Remove computed columns** (for example product total and bill total), because they can be recalculated later.
4. Assign a **primary key** to the table. For this example the composite key **order id + product id** is enough; customer id could also be added as a composite key but is not required, because an order id is unique irrespective of the customer.

A table prepared this way is the **unnormalized form**, and you are now 'ready for normalization'. Note: this is the most complicated preparation work; a student correctly pointed out the need for an exact primary key.

Illustrative unnormalized table (sample values are mine, built from the lecture's customer / order / product example; one row per product in an order):

order_id	order_date	delivery_date	customer_id	customer_name	street	city	pin	product_id	product_name	qty	rate
1	d1	d2	C1	Nilesh	S1	Pune	411001	101	Notebook	4	10
1	d1	d2	C1	Nilesh	S1	Pune	411001	102	Pencil	2	5
1	d1	d2	C1	Nilesh	S1	Pune	411001	103	File	1	20
2	d3	d4	C1	Nilesh	S1	Pune	411001	101	Notebook	2	10

2.2 Why normalize? The three anomalies

A student asked: 'This table is fine and I can insert, edit and delete records in it; why normalize?' The answer: an unnormalized single table suffers from **insert, update and delete anomalies** ('anomaly' means abnormal or wrong behaviour).

Anomaly	What happens	Example from the lecture
Insert anomaly	Some data must be inserted repeatedly (redundant data).	The customer's details are entered again for every product of an order (three products, three entries), and again for every new order the same customer places.
Update anomaly	The same change must be made at multiple records.	The customer changes the delivery address (for example from an office address to another address); you have to modify it in three or four places, not one.
Delete anomaly	Undesired data may be deleted along with what you wanted to delete.	The customer cancels an order; the rows to delete also contain the customer's data, so the customer's information is lost and must be entered again next time, although only the order was cancelled.

Instructor's explanation
The delete anomaly is the worst: the **order** is cancelled, not the **customer**. We want to run our business and we must not lose the customer data just because one order was cancelled. Redundancy (the same data again and again) is the root of all three problems.

3. The Normal Forms

Instructor's explanation
Practically speaking we need only **1NF, 2NF, 3NF and at most BCNF (Boyce-Codd Normal Form)**. Theoretically there are nine normal forms (the instructor said so), but real projects rarely go beyond 3NF or BCNF. These are not the bookish definitions; the lecture teaches the practical way. The instructor also said many learners get confused by the English of the slides; the flow shown on an example is what matters.

Normal form	Steps on the slide	What it removes / ensures
1NF	Steps 1 to 3: remove repeating groups into a new table; key elements become the primary key of the new table; add the primary key of the original table to the new table to give a composite primary key.	Repeating groups (data that repeats for each row)
2NF	Steps 4 to 6: only tables with a composite primary key are examined; columns not dependent on the entire primary key are removed into a new table; their key elements become the primary key of the new table. Repeat for all such tables.	Partial dependency (a column depending only on part of the key)
3NF	Steps 7 to 9: only non-key elements are examined for interdependence; interdependent columns not directly related to the primary key are removed into a new table; the key element becomes the primary key of the new table and is also kept in the original table for the relationship.	Dependency between non-key columns (transitive dependency)
BCNF (optional; some call it 4NF)	Create a separate table for 'well-known' data so that the end user enters minimum data.	Data entry inconsistency; improves data consistency

3.1 First Normal Form (1NF)

Instructor's explanation
Look at one order: which data is repeated? The **customer details** and the **order details** (order date, delivery date) are repeated for every product of the order. Take that repeated part out into a new table (here 'customers and orders'). Leave the rest of the data (product information) in the original table. The new table's key element (order id) becomes its primary key. Then **add the primary key of the new table (order id) to the original table too** to create a composite primary key (order id + product id). It is fine to repeat one column (the key) instead of repeating the whole data.

Table	Columns	Primary key
Customer-orders (new)	order_id, order_date, delivery_date, customer_id, customer_name, street, city, pin, phone	order_id
Order-products (original)	order_id, product_id, product_name, qty, rate	order_id + product_id (composite)

After 1NF there are no repeating groups and we get a **one-to-many** relationship between the two tables (one order has many product lines).

Why the order_id must be also in the second table: without it you could never tell **who ordered** product 101 four times and who ordered it two times; the key links the product lines to their order.

Also check whether any other data repeats within one transaction; if so, separate it again by repeating the same steps.

3.2 Second Normal Form (2NF)

Instructor's explanation
Rule 1: **only tables with a composite primary key are examined**; do not touch tables without a composite key (set them aside). Rule 2: columns that are **not dependent on the entire primary key** must be removed into a new table; the part of the key they depend on becomes the primary key of the new table (and stays in the original table for the relationship).

Examine the order-products table (key: order_id + product_id):

Column	Depends on	Decision
product_name	Only product_id (a notebook is a notebook for every order; it does not depend on the order)	Move to a new **Product** table
qty	Both order_id and product_id (4 notebooks in one order, 2 in another; the same customer may buy a different quantity another day)	Stays in the order-products table
rate	Only product_id in this example (fixed rate for every customer, any quantity)	Move to the Product table (see the note below)

Instructor's explanation
Rate depends on **user requirements**. If your requirement is one fixed rate per product, put rate with the product. If your shop gives different rates for different customers or quantities (for example 18 rupees each for four notebooks and 20 for two), rate depends on the order as well, so it must stay in the order-products table. Both designs are right; it depends on the requirement. (The instructor's own example uses a fixed rate.)

Table after 2NF	Columns	Primary key
Customer-orders	order_id, order_date, delivery_date, customer_id, customer_name, street, city, pin, phone	order_id
Order-products (order details)	order_id, product_id, qty	order_id + product_id
Product (new)	product_id, product_name, rate	product_id

Here the relationship between orders and products is **many-to-many**: one order can have many products, and one product can be bought in many orders. The order-products table is the bridge between them. Repeat steps 4 to 6 for any other table with a composite key.

3.3 Third Normal Form (3NF)

Instructor's explanation
Now only the **non-key elements** are examined, for **interdependence**, in all tables. Order date and delivery date are related to each other and both depend on order_id (delivery cannot come before the order; delivery may take 3, 5 or 10 days, but it is after the order date), so they stay with the order. But customer name, street, city, pin and phone are related to each other and depend on **customer_id**, not on order_id. Your name does not change from one order to the next. So they must go to a new **Customer** table with customer_id as primary key, and customer_id is kept in the orders table **for the relationship** (so we can tell which customer an order belongs to).

Final tables after 3NF	Columns	Key
Customer	customer_id, customer_name, street, city, pin, phone	customer_id (PK)
Orders	order_id, order_date, delivery_date, customer_id	order_id (PK); customer_id (FK)
Order details	order_id, product_id, qty	order_id + product_id (composite PK)
Product	product_id, product_name, rate	product_id (PK)

Relationship	Type
Customer to Orders	One customer has many orders
Orders to Order details	One order has many order lines
Product to Order details	One product appears in many order lines
Orders to Products (through Order details)	Many-to-many

Instructor's explanation
Normalization takes much less time than requirement gathering. The 3NF result is **more than sufficient for any project**; there is hardly a transaction that needs anything beyond 3NF.

3.4 Boyce-Codd Normal Form (BCNF)

Instructor's explanation
BCNF is about **data consistency**: making sure the user does not enter wrong data. Example: a customer enters street 'Janpath' (a street in Delhi), city 'Pune' and pin '40027' (a Mumbai pin code, even missing a digit): this order can never be delivered. To reduce such errors, make the end user enter the **minimum data**. City and pin code are **well-known data** (decided by the government, not by your project), so store them in advance in a separate table (for example **city_pin**) and ask the user to enter only the pin code; the city then follows from it. The pin code is the primary key of that table (it is never repeated), and the customer table keeps the pin code. One pin code can have many customers: a one-to-many relationship.

Table	Columns	Key
City_pin	pin, city	pin (PK)
Customer	customer_id, customer_name, street, pin, phone	customer_id; pin is an FK to City_pin

BCNF is **optional**: your project will still run without it, but doing it is a good practice because it reduces data-entry errors (data consistency problems). It is mandatory in no project. (Companies avoid risk by taking as little data as possible from the user; the rest is stored in advance.)

3.5 Practical statistics from the lecture

- Even the biggest student project hardly has more than 20 to 30 transactions in total, and you usually normalize **at most about 10**, not one for every small screen.
- **Always begin with the biggest transaction** (the screen/transaction with the most data). Find the top 10 transactions among your 20-40 screens. The data of the smaller ones is usually already normalized as part of the bigger ones.
- The instructor's observation (not his own project count but across many projects he has seen): of ten transactions, about two or three finish in 1NF itself, about four or five finish by 2NF, and the rest finish in 3NF. No transaction needs anything after 3NF.
- Example of why the data is already covered: in a bank, the account-opening data (name, address, phone, e-mail, nominee) is given once; a deposit or withdrawal slip needs only the account number.
- A bad requirement analysis gives a wrong UI design, which gives wrong normalization. Most 'normalization problems' students bring are really requirement-analysis problems. Roles matter too: besides the customer, think about the owner (dashboard reports), vendors who update products, shipping/warehouse staff, managers (policy decisions); draw a use case diagram first (as in the library management example: member, librarian, owner).

Additional note (not from the lecture)
The percentages are the instructor's experience, not a rule; treat them as a feel for how quickly most transactions reach 3NF.

4. Effect of Normalization on Queries and Denormalization

After normalization the data is scattered across many tables. Storage is efficient and data is organised, but **some queries become slower and more complicated**. Sample queries (the instructor asked for them as homework; they are about the e-commerce tables above):

Query	Tables needed after normalization
Find the customer with the highest order in the last month	Four tables joined: customers, orders, order details and product (the bill is calculated as qty x rate)
Find the total sale in Mumbai for the last month	All five tables joined (including city/pin), plus GROUP BY
A dashboard showing sales per city (the third query)	Simple in idea, similar joins and grouping (the transcript is unclear on this one)

Instructor's explanation
Why is the data scattered? Because of normalization (split table, split table). The good side: storage is efficient and organised. The bad side: **some queries take much more time** and writing them becomes complicated. A **view** can simplify the query (it is not materialised), but it does not improve execution time, because internally the join still runs. **Indexes** can speed things up, but with millions of records (for example a big sale month) one query can still take 30 to 40 minutes. Do not look for shortcuts to make the query easy; a difficult query is part of the job (companies pay more for difficult work). But we may **compromise on a normalization rule** to speed up execution.

4.1 The denormalization compromise

Remember rule 3 of 'getting ready for normalization': remove computed columns (product total and bill total) because they can be recomputed. To speed up the queries, **add the bill total back into the Orders table** (spend a little storage):

Query	Before (fully normalized)	After adding bill_total to Orders
Highest order (bill)	Joins over three or more tables to compute qty x rate	No join: take MAX(bill_total) from Orders
Customer with the highest order	Four tables	Two tables: customers and orders
Total sale in Mumbai for the last month	Five tables	Three tables (customers, orders, city/pin)

Instructor's explanation
Breaking a rule of normalization deliberately is called **denormalization**. It is done only **if it is required** (for performance of certain analytical queries), not for every project. A student asked: 'If we denormalize later anyway, why normalize at all?' The answer: normalization removes the insert, update and delete anomalies; then, once the requirements (and their slow queries) are known, you denormalize selectively where it is needed. Do not change your table structure many times: freeze the requirements in advance (reports for the owner included). If you did not know a report would be needed, denormalizing later is much more trouble.

4.2 Normalization vs denormalization

Point	Normalization	Denormalization
Aim	No redundancy; efficient storage; no anomalies	Faster queries (fewer joins)
Tables	More tables	Fewer joins; some data duplicated or pre-computed
Queries	Joins across many tables; complicated and slower	Simpler and faster
When	Always as the base design	Only when requirements need it

Common mistake
Do not memorise the normal forms as definitions. Gather requirements first, design the UI, collect the data of each transaction in one place, and only then normalize step by step. Also ask 'depends on what?' for every column; do not ask hypothetical questions.

5. MySQL Architecture and Storage Engines

5.1 Server, client and connectors

Instructor's explanation
The MySQL server (**mysqld**) is the server; the **mysql command line client** (CLI, command line interface) that you have been using from the command prompt is only one kind of client. All your programming so far was done on this client. The **MySQL shell** is available from version 8.0 onward (before that there was only the mysql CLI). The same server can also be accessed from applications through **MySQL connectors** (also called drivers): Java, Node.js, .NET, C#, Python and others. The connectors for each language will be taught in the respective module (for example the MySQL JDBC driver is the MySQL connector for Java).

Whatever the source of a query (CLI, shell or an application), **every query finally reaches the MySQL server**. This is the client-server architecture seen in the introduction.

5.2 What happens inside the server

1. **SQL interface**: the entry point for SQL. (We have studied the SQL side; the NoSQL side of MySQL, a JSON document store, is not in the syllabus.)
2. **Parser**: every query goes first to the parser, which checks the **syntax**: is SELECT spelt correctly, are column names correct, is FROM present, is the order GROUP BY, HAVING, ORDER BY right, is the join written properly?
3. **Query translation**: the query is converted into an **execution plan**. Here the server also checks **privileges** (object privileges): you may be selecting, but do you have SELECT permission on that database; updating, but do you have UPDATE permission? (Recall GRANT and REVOKE, object-level and system-level privileges.)
4. **Optimizer**: produces the **most efficient execution plan** using query access paths, statistics and indexes, and choosing among algorithms (the instructor recalled names such as nested loop and materialization seen earlier).
5. **Cache**: during optimization the server also checks whether this query was executed recently; if the result is cached it is returned immediately instead of reading the hard disk and doing the processing again. The instructor compared it to the idea of a deterministic function, whose result can be cached; here the result of the whole query is cached.
6. **Storage engine**: the plan only says what to do ('read from disk, take this data, sort, group, operate'). The actual execution is done by the **storage engine**, which manages how tables are stored on the hard disk, how they are read, sorting, grouping, joins, indexes (logically an index is a table, physically a data structure such as a B-tree or hash), memory use, and clustering of indexes.

Additional note (not from the lecture)
The query cache was removed from MySQL in version 8.0, so on current versions the 'cache' step does not exist in the same way. The other steps are as described.

5.3 Pluggable storage engines

Instructor's explanation
A special feature of MySQL (the instructor had not seen it in many RDBMS): it does not have a single storage engine; it has **pluggable storage engines**, and the choice is yours. When MySQL was created (by Michael Widenius, 1995, named after his daughter My) it had only one engine, **MyISAM**. Later, as RDBMS requirements grew (transactions and more), a new engine, **InnoDB**, was created. Since version 5.5 or 5.6 InnoDB is the **default engine**, so every table you have created so far is an InnoDB table without you knowing it.

To see the engines: `SHOW ENGINES;` (the engine list includes MEMORY, InnoDB, CSV, MyISAM, Archive, BlackHole, Federated and others; the exact list depends on the install). To see which engine a table uses: `SHOW CREATE TABLE emp;`. You cannot (and need not) change the engine of the whole server; you can specify it per table **while creating** the table, in the table options of CREATE TABLE (for example ENGINE = MyISAM).

```
SHOW ENGINES;
SHOW CREATE TABLE emp;            -- shows ENGINE=InnoDB for default tables

CREATE TABLE sample (id INT, name VARCHAR(20)) ENGINE = MyISAM;
```

Engine	Notes from the lecture
InnoDB	Default; transactions; row-level locking; foreign keys; reliable (transaction log, auto recovery / journaling). More than 90% of cases use InnoDB. A whole InnoDB table is kept in a single file (.ibd).
MyISAM	The original engine. Fast for some read-heavy queries; **no transactions**, only table-level locking, **no foreign keys**. Each table is stored in three files: table definition (.frm), data and index.
MEMORY	Tables kept only in memory, not on disk; for temporary data and caching where speed matters and permanence is not needed.
CSV	Table stored as a CSV (comma separated values) file that can be opened in Notepad; its performance and security are the worst; used sometimes for data backup.
Archive, BlackHole, Federated	Other installed engines (listed in class; not explained).
NDB (NDB Cluster)	Network database: distributes data across multiple machines when the data is too large for one server; a DBA job. 'Distribution independence' relies on it.
Performance Schema / Information Schema	Separate internal engines for metadata and performance data. The information_schema 'tables' are not InnoDB; they are generated by the server.

5.4 InnoDB vs MyISAM

Point	InnoDB	MyISAM
Locking	**Row-level** locking (when the table has a primary key / index, as shown earlier)	Only **full table-level** locking
Referential integrity	Supports **foreign keys** and relationship constraints	No foreign key support
Transactions	Supports **commit and rollback** (ACID transactions)	No transaction support: commit and rollback accepted as commands but do not work; each DML is auto-committed
Reliability	Uses the **transaction log** for automatic recovery (journaling)	No such recovery; less reliable
Speed	Slower on some select queries	Often faster for read-heavy analytical SELECTs when transactions are unimportant
Files	One file per table (.ibd)	Three files per table (definition, data, index)
Usage	Default; more than 90% of cases	A few read-mostly cases

Instructor's explanation
InnoDB is the default and 'more than sufficient' for your project. If you need foreign key constraints you have no choice other than InnoDB. MyISAM was faster in some select-heavy cases, which is why some people still prefer it for tables with few inserts, updates and deletes but many selects. (Because MySQL with MyISAM has no relational features, people call it a DBMS, not an RDBMS.)

Common mistake
A student asked whether MyISAM 'contains' rollback and commit. The commands do not give an error but they do nothing, because every DML operation is auto-committed. Changing the engine changes the behaviour of the same queries.

6. Temporary Tables

Instructor's explanation
A **temporary table** is like a view in being a derived table, but it is **materialised**: it is actually created in **memory**, not in a hard disk file, and its data is stored there. It is called temporary because its **scope and life are limited to the current user session**: you create it in your session, it is not visible from another login (even on the same server), and it is **dropped automatically when you exit** the session. (MySQL specific; it may not exist in every RDBMS.)

6.1 When is it useful?

Instructor's explanation
You have a heavy query joining five or six tables, and you need to run it again and again within one day (say 200-300 times), not every day. A view would only simplify the query; the join would still run each time. Instead, run the join once into a temporary table; further operations then run on the temporary table, which is much faster, because no join is executed again, only the later GROUP BY etc.

6.2 Demo (departments and employees)

Normal query: count the employees in each department and show the department name (a left join so that a department with no employees is also shown):

```
SELECT d.dname, COUNT(e.empno) AS cnt
FROM dept d
LEFT JOIN emp e ON d.deptno = e.deptno
GROUP BY d.dname;

Output:
dname        cnt
ACCOUNTING     3
RESEARCH       5
SALES          6
OPERATIONS     0
```

A view only hides the join (it is not materialised, so performance is unchanged):

```
CREATE VIEW v_emp_dept AS
SELECT d.dname, e.empno
FROM dept d LEFT JOIN emp e ON d.deptno = e.deptno;

SELECT dname, COUNT(empno) FROM v_emp_dept GROUP BY dname;   -- same result; join still runs inside
```

A temporary table runs the join once and stores the result:

```
CREATE TEMPORARY TABLE t_emp_dept AS
SELECT d.dname, e.empno
FROM dept d LEFT JOIN emp e ON d.deptno = e.deptno;

SELECT dname, COUNT(empno) FROM t_emp_dept GROUP BY dname;   -- no join, only GROUP BY on 15 rows
SELECT * FROM t_emp_dept;                                    -- the data is really there
```

Observations from the demo: after creation SHOW TABLES does **not** list the temporary table (it is hidden), yet it can be queried and returns the data. After exiting and logging in again the same query gives the error that the table does not exist, because it was auto-dropped when the session ended. You can recreate it any time (creation takes time when there are millions of records, but queries on it are then very fast). Manual removal:

```
DROP TEMPORARY TABLE t_emp_dept;
```

Point	View	Temporary table
Stored data?	No, not materialised	Yes, a real table with its own data (in memory)
Connection to the base tables	Always reads the base tables; changes there are seen	Only at creation time; after that it is independent (changes in the main table are not reflected, and changes in the temp table do not go to the main table)
Speed for repeated heavy queries	No gain (join runs each time)	Faster (join done once)
Lifetime	Until dropped	Current session only; dropped automatically on exit
Visible in other sessions?	Yes	No
DML allowed?	Only if simple view	Yes, but it affects only the temp table

Additional note (not from the lecture)
MySQL may keep large temporary tables on disk rather than purely in memory; the instructor described them as in-memory tables (the usual simplification). Temporary tables may be dropped explicitly with DROP TEMPORARY TABLE, which is safer than DROP TABLE.

7. Codd's Rules

Instructor's explanation
Codd's rules are for people who want to **build their own RDBMS** or understand **why an RDBMS works the way it does**. For an end user or even an administrator they matter little. They are based on **mathematics (set theory)**: the prerequisite is maths, not databases. **E. F. Codd** was a mathematician who proposed the relational model around 1970 and worked at IBM through the 1980s with a team of programmers on the proposal. A system should follow these rules to be called an **RDBMS**; otherwise it is only a DBMS (any system with insert, update, delete, select). But the rules are **ideal rules** and the real world is not ideal: no RDBMS follows all of them. Even the leading RDBMS, such as Oracle, implements only about **eight** of them successfully. Why? The rules come from mathematics; implementation is programming, and real requirements force compromises.

Codd actually made **one** rule, the **Foundation Rule (Rule 0)**. People did not understand it, so he made the other 12 rules to explain it.

Rule	Statement (as explained)	Link to what we studied / remarks
Rule 0: Foundation rule	A system must be able to manage databases entirely through its relational capabilities. All data, metadata and information must be kept in relational (tabular) form.	The basic rule; the others explain it
1. Information rule	All information in the database (data, metadata, configuration, settings, security information, indexes) is represented explicitly at the logical level in exactly one way: values in **tables**.	DESC output, SHOW GRANTS output, index information are tabular
2. Guaranteed access rule	Every value must be reachable by **table name + primary key value + column name**.	Example: a salary of 1500 is meaningful only as 'salary of the employee with empno 7844 in the emp table'. The name of a person cannot identify a row because names repeat
3. Systematic treatment of NULL	NULL means missing or not-applicable information; it is handled in a **systematic way**, independent of data type; not as zero or blank string.	IS NULL, IS NOT NULL, IFNULL, COALESCE-type functions
4. Dynamic online catalog based on the relational model	The catalog (metadata: structure of data, procedures, functions, triggers) is itself stored in the relational model and can be queried with the same language.	**information_schema** (from MySQL 5.0 onward): tables, routines, triggers, privileges, engines, events. SELECT * FROM information_schema.tables
5. Comprehensive data sublanguage rule	There must be **one language** for data definition, data manipulation, integrity constraints, transaction management and security.	**SQL** with its five parts DDL, DML, DQL, DCL, TCL. Commands such as SHOW TABLES, SHOW CREATE TABLE and SHOW VIEW are **not standard SQL**; they may differ across RDBMS. SELECT is standard
6. View updating rule	All views that are theoretically updatable must be updatable by the system.	Simple views (few columns) are updatable; views with GROUP BY, computed columns, ORDER BY are not (and not expected to be)
7. High-level insert, update and delete	Insert, update and delete can be applied to sets of rows in one operation.	INSERT with multiple row values; INSERT ... SELECT; one UPDATE for 10,000 books; DELETE FROM books; bulk DML
8. Physical data independence	Applications and terminal activities remain logically unaffected when storage representation or access methods change.	**Not fully implemented in MySQL**: with different storage engines the same query can behave differently (for example ROLLBACK and COMMIT do not work with MyISAM)

Rule	Statement (as explained)	Link to what we studied / remarks
9. Logical data independence	Applications are unaffected when the logical structure changes (table split into more tables, column order changed).	Cannot be automatic; **partially achieved with views**: queries fire on a view, so whatever the base tables look like, the view's columns and types stay the same
10. Integrity independence	Integrity constraints are stored in the catalog, not in the application. Two points: **primary key cannot be NULL** (each component of a composite key too), and a **child row cannot exist without its parent row**.	Foreign keys: a department 50 must exist before an employee is entered in it. Works in InnoDB, not MyISAM. Disabling the FK check for faster restore is a back door
11. Distribution independence	The user and application should be unaffected whether the data is on one server or distributed over many.	NDB Cluster; the **MySQL Router** concept routes queries to the cluster
12. Non-subversion rule	If a low-level language exists, it must not bypass the integrity rules; no **back door** to the data.	Only valid committed rows must be visible. Real RDBMS keep back doors for faster backup and restore; disabling FK checks is one (it compromises rule 10 for performance)

Additional note (not from the lecture)
The instructor numbered them in this order. The remark about Oracle implementing 'eight' rules is his recollection ('I heard'). The original list is Rule 0 plus Rules 1 to 12, as above.

Exam note (instructor): such rules are not asked in exams directly. MCQs on the database module are mostly of two kinds: conceptual questions (normalization and similar) and MySQL specific ones (a function's arguments, a scenario with a given table). Knowing the rules still helps if a question appears. Also: **information_schema can be altered, but you should never modify it**, because it is the online catalog that holds your metadata; tampering with it can stop everything that is running.

8. Other Ways to Write a Join: USING and NATURAL JOIN

The standard way is INNER JOIN with ON (the instructor's tables are the employee and department tables):

```
-- 1. ON clause: full control over the condition
SELECT e.ename, d.dname
FROM emp e INNER JOIN dept d
ON e.deptno = d.deptno;

-- 2. USING clause: the column name must be the same in both tables
SELECT e.ename, d.dname
FROM emp e INNER JOIN dept d USING (deptno);

-- 3. NATURAL JOIN: joins automatically on all columns with the same name
SELECT e.ename, d.dname
FROM emp e NATURAL JOIN dept d;
```

All three give the same result for these tables.

Point	ON	USING	NATURAL JOIN
Which columns?	You write the condition	You name the common column(s); names must be **identical** in both tables	**Automatic**: all columns with the same name in both tables
Operator allowed	=, <>, <, > and more	Equality only	Equality only
Different column names (deptno vs dept_id)	Works	Does not work	Does not work

Point	ON	USING	NATURAL JOIN
Flexibility	Full	Less	Least; typing saved, risk of joining on unintended same-named columns

Instructor's explanation
USING cannot express a 'greater than' or 'less than' condition; so a self-join assignment that used less than or greater than (comparing cities, for example) cannot be written with USING. In both, the column names from both tables must be the same. NATURAL JOIN removes even the need to name the columns: if multiple columns have the same name (for example standard and roll number in a students and marks table), it joins on all of them. The instructor said it is made for **'lazy' people who do not like typing**, that he personally never liked it because it reduces flexibility and readability, and that in this form it behaves as an inner join (no left or right outer join is written with it). If it matches your requirement use it; otherwise do not.

Additional note (not from the lecture)
MySQL does support NATURAL LEFT JOIN and NATURAL RIGHT JOIN, so 'inner join only' is true only of the plain NATURAL JOIN form the instructor demonstrated.

9. Introduction to NoSQL

9.1 Database and the two big categories

Instructor's explanation
A **database** is an organised collection of data, generally stored and accessed electronically from a computer system (a definition taken from Wikipedia). A DBMS performs CRUD operations (create, read, update, delete) and advanced operations. People classify DBMS in many ways, but mainly in two: **RDBMS** (what we studied for the last 20-22 days: Oracle, MySQL, Microsoft SQL Server, Sybase, Ingres, Informix, with SQL as the language) and **NoSQL**.

There is also a newer hybrid category, **NewSQL**: NoSQL-like performance with a SQL interface (SQL-like programming), combining RDBMS and NoSQL ideas. Examples named: HarperDB and VoltDB. They are used mainly for big data and machine learning. (NewSQL is not in the syllabus.)

9.2 History: why NoSQL arrived

1. **1970**: relational model (Codd). **By 1980**: RDBMS commercially available and used.
2. **Internet and the web**: the web (WWW) was ready for use around 1989; at first pages were static and only for sharing information (like Wikipedia); there was no interaction or business.
3. **1991**: Java was created (originally for embedded systems), but it became popular through **Java applets** (interactive programs running in a browser) around 1995; interactivity on the internet was a miracle for people used to static pages.
4. Then people started doing business on the internet: a shopkeeper or business that served a city or state became accessible to the whole world; customers grew from thousands to millions. This was the **data burst**.
5. Data moved from MBs to hundreds of GBs and then terabytes. (Instructor's anecdote: his first computer around 1998-2000 had a 20 GB disk, 128 MB RAM and a 1.44 MB floppy; a selfie today is 2-4 MB.) RDBMS of that time could not handle the speed and amount of growth, and performance problems appeared.
6. **1998**: the term **NoSQL** first appeared (**Carlo Strozzi**): 'I know RDBMS but for a given problem I cannot do it well'; he built a database without the SQL language (still tabular).
7. Many other people were facing the same problem. A **global conference** on non-relational databases (the transcript says 2004; the invitation used the name NoSQL) made the idea popular worldwide, and NoSQL grew fast after it. Today many projects use NoSQL, and many use both.

Additional note (not from the lecture)
Standard references usually give 2009 (a NoSQL meetup in San Francisco) as the event that popularised the term, and 1998 for Strozzi's 'NoSQL' database. The transcript audio gives '2004' for the conference; treat that year with care.

Instructor's explanation
NoSQL is **not 'anti-SQL'**: the full form changed to **'Not Only SQL'**, meaning SQL and beyond. RDBMS has its own strengths and NoSQL has its own limitations; each does better in certain areas. Real projects use a combination.

9.3 Features (the instructor's summary)

Feature	Meaning
Flexible schema (no predefined schema)	In an RDBMS you first create the table, columns and data types. In NoSQL the schema can change over time as requirements change; new things can be added to a running production server with efficiency (the Facebook post example: first only text, later background, replies, location, feelings, tags, memories). Altering a table in production is a poor idea; NoSQL is designed for it.
Unstructured and unpredictable data	Text posts, images, audio and video are unstructured data; programmatically finding what a text post says is hard (AI/ML is needed); RDBMS can store them as BLOBs but slowly. NoSQL handles them better.
High performance, high availability, high scalability	Millions of posts per minute (performance); the application runs 24 x 7 (availability); terabytes of data (a few hundred GB to a few TB handled fluently); economical scaling.
BASE instead of ACID	ACID is the RDBMS transaction guarantee (atomicity, consistency, isolation, durability). BASE: **B**asically **A**vailable (the service runs 24 x 7 at least minimally), **S**oft state (the server state may change over time, flexible schema), **E**ventual consistency (changes are visible to all users after some time, not necessarily at once).
Eventual consistency example	You change your Facebook cover page; friends near you see it immediately but someone in the USA may not see it for a few minutes; eventually everyone sees it.

9.4 Vertical vs horizontal scaling

Instructor's explanation
Take a server with 8 GB RAM that serves 100 clients. Now 20,000 clients must connect. **Vertical scaling (scale up)** means increasing the configuration of the same machine: 8 GB to 80 GB RAM, 100 GB disk to 5 TB, a bigger processor. RDBMS usually scales this way. **Horizontal scaling (scale out)** means buying more machines of the same size and putting them in a network as a **cluster**. NoSQL databases are designed for horizontal scaling and give better performance at lower cost.

Point	Vertical scaling (up)	Horizontal scaling (out)
How	Increase RAM/disk/CPU of one machine	Add more machines to a cluster
Cost	Small steps (8 to 16 GB) are cheap, but big jumps (80 or 800 GB RAM) are very costly; the motherboard may not even support it; cost grows **exponentially** (non-linear)	More economical; cost grows more or less linearly; cloud auto-scaling is a related idea
Usually used by	RDBMS	NoSQL

Additional note (not from the lecture)
The instructor mentioned that cloud platforms (AWS) also give a free 1-2 GB RAM by default and that Kubernetes-based auto scaling is a different topic that he did not cover.

PART II - DAY 23 (Evening session)

The recording begins with a recap: last session covered MySQL, regular expressions, temporary tables and the start of NoSQL. Codd's rules are the foundation for building any RDBMS, even though not all 12 are handled by any RDBMS. Next: the idea behind NoSQL, the CAP theorem, and then MongoDB.

10. CAP Theorem

Instructor's explanation
NoSQL databases are based on the **CAP theorem**. CAP stands for **Consistency, Availability and Partition tolerance**.

Letter	Meaning (as explained)
C: Consistency	Once an operation is done and the data is modified, **all clients see the same data** (what RDBMS transactions guarantee at the end of the transaction).
A: Availability	The system is **always on**. It should not happen that the database is unable to handle your request. Not every piece of data need be accessible, but some server must accept requests and give at least a basic, minimal response, 24 x 7, with no downtime for a good reason.
P: Partition tolerance	If the communication between servers is unreliable (the link is broken), the system should **continue to function**.

10.1 Replica sets and partitions

Instructor's explanation
Real applications such as Facebook have users in India, the USA, Australia and so on. To avoid risk, the huge data is distributed across multiple servers. A set of machines connected together carrying **exactly the same data** is called a **replica set**. Its biggest advantage: if any one machine goes down, clients do not suffer, because the same data is on the others. There can be a replica set for India, another for the USA, another for Australia. People mostly access the nearest one; this structure helps **horizontal scalability** and heavy data flow (millions of posts per minute cannot be handled by one or two machines).

The replica sets are interconnected (an Indian user may have US friends, so connectivity across servers is required). Each individual set of servers is called a **partition**. The links between partitions can break for some time (for example 10 to 15 minutes).

Instructor's explanation
If the link between two partitions is broken, is the whole system stopped? No. The Indian users and the US users can both still use their own partitions; basic services stay accessible (there is only a temporary 'down' of that link). A change made in India may not be visible in the USA for a while (a time lag), and when the link is restored the data becomes visible. That is the point: the system continues to function; this is **partition tolerance**. The visibility is eventual, not immediate.

10.2 You cannot have all three

Instructor's explanation
As with Codd's 12 rules, all three CAP properties cannot be implemented together; something is always compromised. This is also called **Brewer's theorem**: **it is impossible for a distributed data store to simultaneously provide more than two of the three guarantees** (consistency, availability, partition tolerance). Any two are possible; the third is usually not feasible in practice. Real systems pick according to requirements: in some projects consistency matters most, in others availability, in others partition tolerance. This applies to **distributed** stores (multiple machines); on a single machine you need not worry.

Combination	Typical systems (as named in the lecture)	Weak at
C + A	Traditional RDBMS (very good at consistency and availability)	Partition tolerance

Combination	Typical systems (as named in the lecture)	Weak at
C + P	MongoDB, HBase, Redis (good at consistency and partition tolerance)	Availability (somewhat lacking)
A + P	Cassandra, DynamoDB, Riak, CouchDB (the lecture mentions CouchDB, Cassandra, DynamoDB, Riak)	Consistency (usually eventual)

Additional note (not from the lecture)
The instructor mentioned Cassandra on the A+P side and also named Cassandra as a column-oriented store. Most references place Cassandra at A+P, and MongoDB and HBase at C+P, as above. Remember that the placement is a trend, not an absolute label.

11. When to Use NoSQL and When Not

11.1 When NoSQL is a good choice

- **Large amounts of data**: terabytes; handled with multiple nodes (a cluster) for horizontal scalability, not one machine.
- **Heavy read and write operations** (example: a cab company's app collecting location pings from every cab every second, millions of rows per second).
- **Economical scaling**: many cheaper machines instead of one very expensive one.
- **Flexible schema** that evolves over time (the Facebook post example).
- **Social media** applications; **blobs** (audio, video, images) stored on the server and processed heavily; **geospatial analysis** (grouping by geo location, analysing places on a map); **information and log processing**; and **unstructured data** processing (for example estimating a person's bias or leaning from their posts).

11.2 When NoSQL is not a good choice

- Where you need **ACID transactions**: atomicity, consistency, isolation, durability. For financial transactions (a withdrawal must immediately show the same balance at the ATM) the RDBMS is still the best. Consistency is usually the weak zone of NoSQL; there you get 'eventual consistency' instead of strong, tight consistency.
- Where there are **many relations and joins**: a project with many tables joined together. Some NoSQL databases support joins (MongoDB has $lookup), but not many do, and compared with an RDBMS they are weaker.
- Business operations needing strong guarantees. Banking applications often use a mixture: the main financial part in an RDBMS, other speed-oriented areas in MongoDB.

11.3 Hybrid use and project use

Instructor's explanation
In real, large systems it is common to use **multiple databases in one project**: put in the RDBMS what the RDBMS does best and in NoSQL what NoSQL does best. This is called a **hybrid mode**. In the course you will meet the **microservices** architecture, where a large application is broken into small applications; some of them can use MySQL and others MongoDB. Whether your own project uses a NoSQL database is your choice; students rarely do. If you do, you need to study MongoDB more than what is covered in class. (The instructor's own institute website uses an RDBMS for the main site, but the recorded-lecture, quiz and schedule data is in MongoDB.)

11.4 RDBMS vs NoSQL

Point	RDBMS	NoSQL
Data model	Rows and columns in tables, related to each other	Not tabular: key-value, document, column or graph
Language	SQL (ANSI SQL standard used everywhere)	**No standard**; differs by database (each built for specific requirements); no future standardisation expected
Schema	Fixed; decided in advance (changing a production table structure is poor practice)	Flexible; evolves over time
Scaling	Mainly vertical (scale up)	Mainly horizontal (scale out)
Transactions	ACID	BASE; eventual consistency
Joins and relationships	Strong support	Limited; weaker
Support	High level of enterprise support over many years	Many are open source (free); community support; a company may sell commercial support (for example the company behind MongoDB)
Maturity	Older (from 1970) and more mature	Younger (from about 1998); newer ones are maturing
Best for	Enterprise applications, financial transactions, many relations	Huge data, heavy read/write, unstructured data, social media, geospatial, logs

11.5 Migrating from RDBMS to NoSQL (syllabus item)

Instructor's explanation
This is not very relevant for fresher level, but the idea: a project starts in MySQL and later the requirement suggests MongoDB. Approach 1: **export** the tables from MySQL as CSV or JSON, and **import** them into MongoDB (the same tables map to collections). Approach 2: if two MySQL tables must become one combined collection in MongoDB, **create a view in MySQL** that joins them, export that view, and import it into MongoDB. Tools: MySQL export / dump commands and the **mongoimport** command. This helps core developers when a running MySQL project must move to MongoDB.

12. Types of NoSQL Databases

NoSQL databases are all **non-relational** (not in tabular format) and there is **no standardisation**: each was built for specific project requirements, and they differ from each other. Four main categories:

Type	Idea	Examples named
Key-value	Data is kept as keys and values. A key is unique; the value can be of any type. It is the hash table concept in data structures, built as a big **distributed** hash table for the fastest possible search.	Redis, DynamoDB (Amazon's, built after its white paper)
Column-oriented (columnar)	Data is stored column by column, not row by row; good for aggregation and analytics on a few columns.	HBase, Cassandra
Graph-oriented	Data is kept in vertices (nodes) and edges (relationships). Fast for relationship-heavy queries.	Neo4j (used in the instructor's project), TitanDB
Document-oriented	Data is stored as documents in JSON-like format; collections of documents.	MongoDB

12.1 Key-value stores

Instructor's explanation
Think of how a mobile phone stores contacts: you enter a name and a mobile number, and the main purpose is that when you give the name, you get the mobile number quickly. That is key-value storage. In data structures it is a **hash table**: data stored as key-value pairs, and for a given key the value can be searched in the fastest possible way. The key-value database applies the same concept at scale.

Example: key = 'student:1:name', value = 'ABC'; key = 'student:1:address', value = 'Pune'; hobbies, an image (binary), anything. The value can be a string, an array, an object, an image, audio, video, JSON, a blob, an integer, a long. Keys must always be unique. Data is spread across multiple machines (a distributed hash table), and the main use is **very fast searching**.

12.2 Column-oriented databases

Instructor's explanation
In an RDBMS the data is stored row by row on the hard disk (scattered through the disk). Take the query that sums salary per job: **SELECT job, SUM(sal) FROM emp GROUP BY job** over millions of records. You scan every row of the table even though you need only two of its eight columns, and the more you read, the slower the operation. A **column-oriented** (columnar) database stores values of a column continuously: first the names of all rows, then jobs, managers, salaries, commissions, department numbers, and so on. Now only the two needed columns are read; the operation is much faster. That is the beauty of it: better performance while accessing a few columns and aggregations, with huge (terabyte) data.

Caution: SELECT * on all columns will be slow on a columnar store (slower than an RDBMS). Best for business intelligence, CRM, data warehousing, dashboards and reporting, where only a limited number of columns are used. Examples: HBase and Cassandra (HBase, Cassandra, 'Big Table' are given in the Big Data course syllabus).

12.3 Graph databases

Instructor's explanation
A graph is made of **vertices and edges** (graph theory; not a drawing graph). Data is kept in the vertices and relationships in the edges. Example with employees and departments: employee 1, 2, 3 are vertices; department 1, 2, 3 are vertices; an edge 'belongs to' connects an employee to a department; an edge 'reports to' connects an employee to another employee (employee 8 reports to employee 7, employee 2 reports to employee 1). Finding all employees of a department, or whom employee 4 reports to, or 'the manager of the manager of employee 2', is very fast (the instructor compares it to O(1)). In an RDBMS the same needs self joins (two or three self joins, with millions of rows, a kind of Cartesian product) and is complicated and slow.

Used where relationships are central, as in social media sites: each person is a vertex, and friendships are edges. Examples: Neo4j (speed is 'fantastic' but it is designed for specialised requirements and cannot be used everywhere) and TitanDB.

12.4 Document-oriented databases

Instructor's explanation
One **document** contains data in a **JSON** format (or XML). MongoDB is the document database studied next. The terminology maps to the RDBMS ideas you know: what an RDBMS calls a table, MongoDB calls a **collection**; a row is a **document**; columns/values are **fields** (key and its value).

RDBMS	MongoDB
Database	Database
Table	**Collection**
Row (record)	**Document** (a JSON/BSON object)
Column	**Field** (key and value)
Primary key	_id
JOIN	$lookup (in aggregation)
Fixed schema	Flexible schema

13. MongoDB: JSON, BSON, Installation and Architecture

13.1 JSON (JavaScript Object Notation)

Instructor's explanation
JSON stores data as **key-value pairs** inside curly braces. **Keys are always strings**; values can be of different types: integer, string, double, boolean, array, or even another object. Standard JSON requires keys in **double quotes**; the MongoDB shell relaxes this: you can write keys without quotes and it still works (a typing saver). The data looks like a C structure. Full form: **JavaScript Object Notation**.

```
// Book example (simple)
{
    bookId: 1,
    title: "Java Basics",
    author: "Kanetkar",
    price: 450
}

// Resume example (nested object, array, boolean, double)
{
    id: 1,
    name: "Nilesh",
    age: 40,
    hobbies: ["programming", "reading"],      // array: square brackets
    address: {                                 // sub-object
        area: "Katraj",
        city: "Pune",
        pin: 411046
    },
    political: false,                          // boolean
    height: 5.9                                // floating point
}
```

Instructor's explanation
This flexible structure means every resume can have different fields: one with only the basics, another with academics added, another with experience and family relationships. You do not need every field in every document; the schema can evolve over time as needed.

13.2 What is MongoDB?

- A **document-oriented NoSQL database**; data is stored as JSON-like documents, in **binary form (BSON)** on disk.
- Developed in **2007** by a company called **10gen** (the spelling in the transcript is garbled), as part of one of its projects; after use in that project they realised it could be useful to others and made it public in **2009** as **open source** (the source code is on GitHub). Even today almost all developers are from that company (now MongoDB).
- It first appeared in the C-DAC syllabus in 2011 (an old anecdote); MongoDB itself is from 2007; it is much more mature today.
- The server is written in **C and C++** for the highest performance (as are most major databases).
- Design philosophy (from its website): it was **not designed in a lab**; it was built from experience of building large-scale, highly available, robust systems.

13.3 JSON vs BSON (why binary?)

Instructor's explanation
The biggest problem with JSON, XML and plain text is **parsing**: understanding what is written needs a program (like the parser of the MySQL architecture or a compiler's parser), and parsing is a time-consuming task; more parsing means slower performance. So MongoDB takes data from you as JSON but **does not store it as JSON**. It converts it into **binary (BSON: Binary JSON, 'binarization')** and writes that to the hard disk, so the next access does not need parsing. Reading binary is always faster than reading textual data.

13.4 Installation and tools

1. Download the MongoDB MSI installer for Windows (the demo used the 64-bit Community server, version 4.4 as seen in the install path).
2. Choose the **Complete** installation; install **MongoDB as a Service** (it then runs in the background; if the shell says it cannot connect later, open Services (services.msc) and start the MongoDB service).
3. MongoDB **Compass** is an optional graphical tool; it is not needed (a more advanced feature).
4. Add the **bin** folder (under C:\Program Files\MongoDB\Server\<version>\bin) to the **PATH** environment variable (This PC > Properties > Advanced system settings > Environment Variables > Path > Edit > New), exactly as was done for MySQL. Then open a **new** command prompt.
5. Run **mongo** to open the shell. By default **security is disabled** in MongoDB, so there is no user name or password to enter.
6. Press Ctrl + L to clear the screen.

Additional note (not from the lecture)
The demo used the classic `mongo` shell. Recent MongoDB versions replaced it with `mongosh`, and methods such as insert and remove are deprecated in favour of insertOne / insertMany / deleteOne / deleteMany. The lecture's commands are accurate for the version shown.

13.5 MongoDB components

Component	Role (compare with MySQL)
mongod	The MongoDB **server** (like mysqld). Written in C/C++. Runs in the background as a service.
mongo (shell)	The primary **client** (like the mysql CLI). Queries are written in **JavaScript syntax**: *objectName.methodName(parameters)*; you only need that pattern, not full JavaScript. The shell can also run JavaScript script files.
mongofiles / GridFS	A tool to store very large files (MBs or GBs): images, recordings, audio, video.
mongodump / mongorestore	Backup and restore (like mysqldump).
mongoimport / mongoexport	Import and export data (CSV or JSON).
Drivers	MongoDB can be accessed from applications through drivers: Java, Python, Ruby, PHP, Perl, Node.js and others (like MySQL connectors). Your course projects will be on Java or Node.js (full stack); both have MongoDB support.

The client-server architecture is the same as MySQL: queries are written in the client, sent to the server, processed there, and the response is returned to the client.

14. MongoDB CRUD Operations

MongoDB is a DBMS like MySQL, so the functionality is similar (insert, select, update, delete); the syntax differs. The shell is not MySQL: do not compare it with MySQL command by command.

14.1 Databases and collections

```
show dbs              // lists databases (also: show databases)
use dbName            // switch to a database (creates it lazily)
show collections      // lists collections of the current database
```

By default three system databases exist: **admin**, **config** and **local** (like the system databases of MySQL); do not use them for your own data. **use dbname** does not create the database; it only switches. The database is created automatically when you add the first record. Likewise, a collection is created either automatically when you start inserting records, or explicitly with a creation method (such as createCollection). **show dbs** displays sizes like 0.000GB because the real size is a few KB, which does not appear in GB.

14.2 Insert

Each document must be a JSON object, so wrap it in curly braces. Inserting the first document creates the database and the collection.

```
use dbName

db.contact.insert({
    name: "Nilesh",
    email: "nilesh@info.com",
    mobile: "9876543210"
})
// WriteResult({ "nInserted" : 1 })

show collections       // contact
```

Flexible schema demo (illustrative values; the instructor inserted five contacts with different fields in the same collection):

```
db.contact.insert({ name: "Sandeep", email: "sandeep@info.com" })                    // no mobile
db.contact.insert({ name: "Prashant", mobile: "9000000001", address: "Karad" })      // different fie
db.contact.insert({ name: "Sunbeam Infotech", fax: "24260308", website: "sunbeaminfo.com" })
```

All five documents live in one collection even though each has different fields: this is a **flexible schema**. You cannot be forced to give every field in every document.

insertMany inserts multiple documents in one call: pass an **array** (square brackets) of objects.

```
db.contact.insertMany([
    { name: "A", mobile: "111" },
    { name: "B", email: "b@x.com" },
    { name: "C", mobile: "333", email: "c@x.com" }
])
// acknowledged: true, insertedIds: [ ObjectId(...), ObjectId(...), ObjectId(...) ]
```

> **Instructor's explanation**
> A student asked why a single insert does not display the ObjectId while insertMany does. These are two different functions with different output behaviour. In the 2011 version only insert existed; the newer version added insertMany and its additional information.

14.3 The _id field (ObjectId)

Instructor's explanation
Every document must have an **_id** (the primary key; its name is always **_id**, a decision of the MongoDB team). You may give it yourself; if you do not, MongoDB generates an **ObjectId** automatically on the client side. The ObjectId is **12 bytes**, made of several components so that it is unique across many clients: **timestamp** (number of seconds since 1 January 1970, 4 bytes), **client machine id** (3 bytes), **process id** of the client process (2 bytes; two clients on one machine have different process ids), and a **counter** (3 bytes; so documents inserted at the same second by the same process still differ). If you give your own _id, it must be **unique**, or MongoDB gives an error; you must keep that in mind.

```
db.emp.insert({ _id: 7844, ename: "TURNER", sal: 1500 })     // your own _id
```

Maximum size of one document: 16 MB (not of the collection). A typical post is far smaller. Videos, audio and images larger than this go through **GridFS**, which stores them in chunks.

14.4 Find

db.collectionName.find(criteria, projection) returns a **cursor**. The word **db** is a fixed keyword for the current database; the collection name follows; find is a method that returns all documents, which you can then process further.

```
db.emp.find()                // all documents (hard to read)
db.emp.find().pretty()       // formatted output, easier to read
db.emp.findOne()             // only one document
```

To read a lot of documents the cursor offers methods; the lecture used these:

Cursor method	Meaning	Equivalent in MySQL
pretty()	Show the result in a readable format	-
skip(n)	Skip the first n documents	LIMIT offset, count (offset part)
limit(n)	Show only n documents	LIMIT n
sort({column: 1})	Sort; **1 = ascending**, **-1 = descending**	ORDER BY

```
db.emp.find().pretty()
db.emp.find().skip(10)                    // 14 employee docs: skip 10, show the last 4
db.emp.find().limit(4)                    // first 4 documents
db.emp.find().skip(2).limit(4)            // skip 2, then show the next 4 (like LIMIT 2, 4)
db.dept.find().sort({ dname: 1 })         // ascending by dname
db.dept.find().sort({ dname: -1 })        // descending by dname
db.emp.find().sort({ sal: -1 }).limit(3)  // top 3 employees by salary
```

Expected for the last query: the three highest salaries (5000, 3000, 3000 in the lecture's employee data), shown with pretty() if you append it. The sample employee data is the same emp/dept data used in MySQL.

14.5 Projection (which fields to show)

Instructor's explanation
find takes **two arguments**: the first is the **criteria** (like WHERE) and the second is the **projection** (which fields to show). When you give both they must be JSON objects. With no criteria, pass an empty object **{}**. In the projection, **1** means show the field and **0** means do not show it. **_id is shown by default**; to hide it write **_id: 0**. Every document must have an _id, so MongoDB shows it unless you say otherwise.

```
db.emp.find({}, { ename: 1, sal: 1 })                    // _id, ename, sal
db.emp.find({}, { ename: 1, sal: 1, _id: 0 })            // only ename and sal
db.emp.find({}, { ename: 0, sal: 0, job: 0 })            // everything except these three
```

Common mistake
Do not mix 1 and 0 in a projection (some fields 1, some 0). Either list what to show or list what to hide. The only allowed mix is _id: 0 together with 1s.

14.6 Criteria (like WHERE)

```
db.emp.find({ deptno: 20 })                // WHERE deptno = 20   (5 employees)
db.emp.find({ job: "CLERK" })              // strings in quotes; case-sensitive
db.emp.find({ job: "clerk" })              // nothing: case mismatch
```

Matching is **case-sensitive** by default. Field names are case-sensitive too (the shell follows JavaScript). To search case-insensitively or by pattern, use a **regular expression**: put the pattern between two slashes, and an **i** after the closing slash for case-insensitive:

```
db.emp.find({ job: /^clerk$/i })           // case-insensitive match of CLERK
db.emp.find({ ename: /^S/ })               // names starting with S
db.emp.find({ ename: /.+AM.+/ })           // AM somewhere in the middle (not at the start or end)
```

The same regular expression knowledge from MySQL applies: ^ start, $ end, . any character, + one or more, * zero or more, [] range. Only the syntax changed (two slashes instead of REGEXP with quotes). In the lecture the 'in between' example: dot means any character, plus means multiple occurrences, so *.+AM.+* finds names with AM in the middle.

14.7 Operators ($ operators)

Instructor's explanation
For conditions other than equality, use operators that start with a dollar sign. The syntax is nested curly braces: **{ column: { $operator: value } }**. First the column name, then in its own braces the operator and the value.

Operator	Meaning	Example
$gt	greater than	{ sal: { $gt: 2500 } }
$gte	greater than or equal	{ sal: { $gte: 2500 } }
$lt	less than	{ sal: { $lt: 2500 } }
$lte	less than or equal	{ sal: { $lte: 2500 } }
$ne	not equal	{ deptno: { $ne: 20 } }
$eq	equal (optional; plain value works)	{ deptno: { $eq: 20 } }
$in	value in a list (array)	{ job: { $in: ["PRESIDENT", "ANALYST", "SALESMAN"] } }
$nin	value not in a list	{ job: { $nin: ["CLERK"] } }

```
db.emp.find({ sal: { $gt: 2500 } })
db.emp.find({ job: { $in: ["PRESIDENT", "ANALYST", "SALESMAN"] } })    // like WHERE job IN (...)
```

There are many more operators (about five days would be needed to cover all in depth); read the official MongoDB manual on the MongoDB website: each operator is documented with examples. The instructor repeated that the manual is the real documentation, while an online tutorial is not a documentation.

14.8 Combined: top three earners and a limited projection

```
db.emp.find({}, { ename: 1, sal: 1, _id: 0 }).sort({ sal: -1 }).limit(3)
```

This is equivalent to *SELECT ename, sal FROM emp ORDER BY sal DESC LIMIT 3* in MySQL. Method order: find (criteria, projection), then sort, then limit.

14.9 Delete

```
db.dept.remove({ deptno: 40 })            // old function: remove matching documents
db.dept.deleteOne({ deptno: 40 })         // newer: delete the FIRST document that matches
db.dept.deleteMany({ deptno: { $gt: 30 } })   // newer: delete ALL documents that match
db.dept.remove({})                        // empty criteria = delete ALL documents
db.dept.deleteMany({})                    // same result
db.dept.drop()                            // drop the whole collection
```

Instructor's explanation
Old MongoDB (2011) had only **remove**. The newer versions added **deleteOne** (delete the first document matching the criteria) and **deleteMany** (delete all documents matching the criteria). With an empty criteria, all documents are deleted but the **collection remains (empty)**; collections are not deleted automatically. To delete the collection itself use **drop** (like DROP TABLE): the collection disappears from show collections permanently.

There is no transaction and no rollback by default for such operations, so a deleted document cannot be recovered automatically; be careful before deleting. (In class: dept collection had four documents; after remove with deptno 40 only three remained; deleteMany({}) deleted the three; drop removed the collection, leaving only contact and emp.)

14.10 Update

Instructor's explanation
An update needs **two things**: the **criteria** (which documents) and the **changes** (what to do). The changes are written using the **$set** operator (like SET in SQL UPDATE). Syntax: `db.collection.update({ criteria }, { $set: { field: newValue } })`.

```
// MySQL: UPDATE emp SET sal = 1000 WHERE ename = 'JAMES';
db.emp.find({ ename: "JAMES" })                                  // sal is 950
db.emp.update({ ename: "JAMES" }, { $set: { sal: 1000 } })       // Diwali bonus
db.emp.find({ ename: "JAMES" })                                  // sal is now 1000
```

Additional note (not from the lecture)
Newer MongoDB versions provide updateOne and updateMany (and replaceOne), similar to deleteOne and deleteMany. Plain update() changes only the first matching document unless you pass the multi option.

15. Other MongoDB Topics Raised in Class

15.1 Constraints and validators

There are **no constraints by default**: you can add any field of any type anywhere. If you need constraints (like NOT NULL, CHECK in MySQL), MongoDB has **validators** (schema validation). You create a collection with the **createCollection** method and give a validator that describes the rules: the document is an object, which fields are required, the type of each field (name must be a string, year must be an integer, and so on). On every insert the validator checks the document and gives an error if it does not match (for example an integer entered for a string field).

15.2 Indexes

Index type	Notes
Normal (single field)	Like the basic MySQL index
Composite	On several fields
Unique	Values must not repeat
Geospatial	For location-based queries (for example grouping by geographic location)
TTL (time to live)	Documents expire automatically after a set time (for example a record should stay in the database for a fixed period and then be deleted automatically)

MongoDB has five types of indexes. (MySQL's three types were normal, composite and unique, as studied.)

15.3 Aggregation and joins ($lookup)

Aggregate operations are a very important feature (the minimum to learn if you want to do a project in MongoDB). They run through **db.collection.aggregate([...])** with an **aggregation pipeline**: a sequence of stages inside an array, such as group, sort, match (like WHERE) and project. Everything possible in MySQL aggregates is possible here and more. **Joins** are done with the **$lookup** stage, which is part of the aggregation pipeline. MongoDB can do a good amount of joins, but compared with an RDBMS not many NoSQL databases are good at it.

```
db.emp.aggregate([
    { $group: { _id: "$deptno", totalSal: { $sum: "$sal" } } },
    { $sort: { totalSal: -1 } }
])
```

Additional note (not from the lecture)
The aggregate example is mine: the lecture only named the stages. $group computes the total salary per department in this sketch.

15.4 Transactions

NoSQL databases generally have no ACID transactions, so the commit and rollback commands do not exist in the classic way. In MongoDB the guarantee given in the lecture is: **a single document is always updated atomically**: if one user is updating it, another user cannot update the same document at the same moment. START TRANSACTION and similar commands from MySQL are not there; therefore a deleted record cannot be rolled back.

Additional note (not from the lecture)
Newer MongoDB versions (4.0 onward) do add multi-document transactions; the lecture describes the classic behaviour.

15.5 Stored procedures, functions, triggers

MongoDB does **not** have stored procedures and triggers as in MySQL. A concept exists to pass JavaScript functions to the server (anonymous functions), but it has nothing to do with stored procedures or triggers.

15.6 Storage engines in MongoDB

The old engine was **MMAPv1** (instructor's 'MMAP API') and the new, current engine is **WiredTiger** ('wired tiger'). In the lecture's words, it is not as pluggable as MySQL's engines were described, and the instructor said he was not sure of the pluggable detail.

15.7 GridFS (large files)

Files in MBs or GBs (images, videos, audio) are stored in **GridFS** in chunks; the basic document stays within 16 MB. A video uploaded to a social site goes to GridFS, not into the post document. The **mongofiles** tool stores and retrieves such files.

15.8 Running a script file and importing

MongoDB has no SOURCE command like MySQL, but you can run a JavaScript file with the shell:

```
mongo dbName path\to\script.js        // runs the commands in the file against dbName
mongoimport ...                         // imports CSV or JSON (see its documentation)
```

15.9 Q&A summary from the lectures

Question	Answer
Is MongoDB case-sensitive?	Yes: collection names, field names and string values (use /regex/i for case-insensitive search).
Are quotes needed around keys?	Standard JSON requires double quotes; the MongoDB shell accepts keys without quotes.
Why does show dbs show 0.000GB?	The data size is only a few KB, which does not appear in GB.
How does Facebook-like data fit in a 16 MB document?	A post is much smaller; videos, audio and images go to GridFS in chunks.
Is 'sizeof document' the size of the collection?	No. 16 MB is the maximum size of ONE document.
Can we use MySQL and MongoDB together?	Yes. Microservices architecture allows different services to use different databases.
How do we know which storage engine MongoDB uses?	Old: MMAPv1; current: WiredTiger.
Will NoSQL come in the exam?	Yes: the theory of NoSQL (concepts, CAP, types, features) is asked in MCQs far more than the MongoDB practical.
Why not just use CSV engine?	CSV has the worst performance and poor security.
Can you recover a deleted document?	Not automatically: no rollback. Take care before deleting.

16. Revision Material

16.1 Summary of all major topics

- **Normalization** removes redundancy and the **insert, update and delete anomalies**. Do it per transaction, starting with the biggest ones; requirement analysis and UI design come first.
- **Getting ready:** list all fields of a transaction, one table, data types, remove computed columns, assign a primary key (often composite).
- **1NF:** remove repeating groups into a new table; composite key in the original. **2NF:** only composite-key tables; move columns not depending on the whole key. **3NF:** move interdependent non-key columns into a table keyed by their determinant; keep that key for the relationship. **BCNF:** well-known data (city and pin) in a separate table; optional; improves data consistency.
- Normalization increases the number of joins; **denormalization** deliberately compromises a rule (for example store the bill total) for speed. Views do not speed up queries; indexes help.
- **MySQL architecture:** clients and connectors, SQL interface, parser (syntax), translation (privileges, execution plan), optimizer, cache, storage engine. **Pluggable storage engines:** InnoDB (default; row-level locking, FKs, transactions, reliable) vs MyISAM (original; table locks; no FKs/transactions; fast reads); also MEMORY, CSV, Archive, BlackHole, Federated, NDB.
- **Temporary table:** materialised, session-scoped, hidden, auto-dropped; speeds repeated heavy queries; independent after creation. A **view** is not materialised.
- **Codd:** foundation rule + 12 rules; ideal, not fully implemented; MySQL partly (information_schema, SQL, views, FK in InnoDB).
- **Joins:** ON (flexible), USING (same column name, equality only), NATURAL JOIN (automatic, least flexible).
- **NoSQL:** Not Only SQL; non-tabular; flexible schema; unstructured data; BASE; horizontal scaling; no standard language. **CAP:** consistency, availability, partition tolerance; only two of three. **Types:** key-value, column, graph, document.
- **MongoDB:** document store (JSON as BSON), server mongod, client mongo (JavaScript syntax). Collections and documents; _id (ObjectId, 12 bytes); max document 16 MB; insert / insertMany; find with criteria and projection; $gt etc.; regex; skip, limit, sort; remove / deleteOne / deleteMany / drop; update with $set; validators; five index types; aggregation and $lookup; GridFS.

16.2 Important definitions and syntax

Term / Syntax	Meaning
Anomaly	Wrong behaviour caused by redundancy: insert, update, delete
Composite primary key	A primary key made of two or more columns
Denormalization	Deliberately breaking a normalization rule for performance
Storage engine	Component that stores and reads tables (InnoDB, MyISAM ...)
Temporary table	Session-only materialised table: CREATE TEMPORARY TABLE t AS SELECT ...
Information schema	Online catalog (metadata) in tabular form
`emp NATURAL JOIN dept`	Join on all same-named columns
BASE	Basically Available, Soft state, Eventual consistency
CAP	Consistency, Availability, Partition tolerance
Replica set	Machines carrying the same data
`show dbs; use db; show collections;`	Mongo shell navigation
`db.coll.insert({...}) / insertMany([...])`	Insert one / many
`db.coll.find(criteria, projection)`	Query
`sort({f:1}) skip(n) limit(n) pretty()`	Cursor methods
`{ f: { $gt: 5 } }, { f: { $in: [..] } }`	Operators
`db.coll.remove({}) / deleteMany({}) / drop()`	Delete documents / collection
`db.coll.update({c}, { $set: {f: v} })`	Update

16.3 Key differences between related concepts

Concept A	Concept B	Key difference
1NF	2NF	Removes repeating groups vs removes partial dependency on a composite key
2NF	3NF	Column depends on part of the key vs non-key columns depend on each other
3NF	BCNF	Dependency rule vs data consistency using pre-stored well-known data
Normalization	Denormalization	Redundancy removed vs redundancy added deliberately for speed
View	Temporary table	Not stored vs stored in memory for the session
InnoDB	MyISAM	Row locks, FKs, transactions vs table locks, no FKs, no transactions
ON	USING / NATURAL	Full control vs same-name columns / automatic
RDBMS	NoSQL	Fixed schema, ACID, vertical vs flexible schema, BASE, horizontal
Vertical scaling	Horizontal scaling	Bigger machine vs more machines (cluster)
Key-value	Document	Opaque value under a key vs structured JSON document with fields
JSON	BSON	Text, needs parsing vs binary storage, faster
remove / deleteMany	drop	Delete documents (collection stays) vs delete the collection
insert	insertMany	One document vs an array of documents

16.4 Common mistakes to avoid

Mistake	Why it is a problem	Correction
Normalizing without requirement analysis and UI design	Wrong tables	Gather requirements and design the UI first
Normalizing every small screen	30-40 times the work	Start with the biggest transactions; usually about 10
Keeping computed columns in the unnormalized form	Redundant, can be recalculated	Remove them (denormalize later only if needed)
Putting rate with the product when rates differ per order	Wrong dependency	Decide using the requirement
Expecting a view to speed up a query	View is not materialised	Use indexes, a temp table or denormalization
Using MyISAM and expecting rollback or foreign keys	Not supported	Use InnoDB
Mixing 1 and 0 in a Mongo projection	Not allowed (except _id)	Use only 1s or only 0s
Using find('clerk') case mismatch	Match is case-sensitive	Use /clerk/i
Mongo _id duplicate	Error	Keep custom _id unique or let MongoDB generate it
Assuming all documents in a collection have the same fields	Schema is flexible	Validate or check fields in code
Modifying information_schema	Can stop the server's metadata	Never alter it
Expecting rollback in MongoDB after deleteMany	No transactions by default	Be careful; take backups

16.5 Interview questions and answers

Items marked (extra) go beyond the lecture.

Question	Answer
What is normalization and why is it needed?	Organising tables to remove redundancy and the insert, update and delete anomalies.
Explain the three anomalies with an example.	Customer data repeated per product (insert), address change in many rows (update), losing customer data when an order is cancelled (delete).
What is 1NF / 2NF / 3NF / BCNF?	1NF: no repeating groups; 2NF: no partial dependency on a composite key; 3NF: no dependency between non-key columns; BCNF: stricter, with well-known data in a separate table.
What is denormalization and when is it used?	Intentionally breaking a normalization rule to make queries faster; used only when the requirements need it.
Explain MySQL architecture.	Clients/connectors, SQL interface, parser, translation to an execution plan with privilege check, optimizer, cache, pluggable storage engine.
What is a storage engine? Name some.	The component that stores and retrieves table data: InnoDB, MyISAM, MEMORY, CSV, Archive, NDB.
InnoDB vs MyISAM?	InnoDB: row-level locking, foreign keys, transactions, crash recovery; MyISAM: table locks, no FKs or transactions, sometimes faster reads.
What is a temporary table?	A table created in memory for the current session that is dropped automatically at the end of the session.

Question	Answer
View vs temporary table?	A view stores no data and re-runs its query; a temporary table stores the result once.
What are Codd's rules?	Foundation rule plus 12 rules defining what a relational DBMS must satisfy.
What is NATURAL JOIN?	A join that automatically uses all columns with the same name.
What is NoSQL?	'Not Only SQL': non-relational databases with flexible schema, horizontal scaling and BASE.
Explain the CAP theorem.	A distributed store can guarantee at most two of consistency, availability and partition tolerance.
What are BASE properties?	Basically Available, Soft state, Eventual consistency.
Vertical vs horizontal scaling?	Bigger machine vs more machines; NoSQL prefers horizontal and costs less.
Types of NoSQL databases?	Key-value, column-oriented, graph, document.
What is MongoDB?	A document-oriented NoSQL database that stores JSON as BSON.
Terminology mapping MySQL to MongoDB?	Table to collection, row to document, column to field.
Maximum size of a MongoDB document?	16 MB.
What is ObjectId?	A 12-byte automatically generated _id made of timestamp, machine id, process id and counter.
How do you do a join in MongoDB?	With the $lookup stage in an aggregation pipeline.
(extra) What replaced the mongo shell?	mongosh.

16.6 Practice exercises (solutions not included)

Easy

1. List the three anomalies with an example for each.
2. State the rule of 1NF, 2NF and 3NF in one sentence each.
3. Open the mongo shell, switch to a new database and insert one document into a new collection. Which two things were created?

Medium

1. Take your own project's biggest transaction (for example a library issue or an online order). Write the unnormalized table, then normalize to 3NF and list tables, keys and relationships.
2. Decide whether rate should be in the product table or the order-details table for a shop that gives quantity discounts. Justify it.
3. Find the engine of an existing table with SHOW CREATE TABLE and create one table with ENGINE = MyISAM; try a foreign key on it.
4. Create a temporary table for a join of emp and dept and compare queries on it with the same queries through a view.
5. In MongoDB, insert the five contacts with different fields and then use find with a projection to show only names.

Challenging

1. Write the three queries of the e-commerce homework (highest order customer, total sale in a city, sales per city) on the normalized tables; then add bill_total to Orders and rewrite them. Compare the number of joins.
2. For a social media feature, explain why a document or graph database may fit better than an RDBMS, and what you would give up (use CAP and ACID in your answer).
3. Write MongoDB queries for: employees with salary above 2500 sorted by salary descending showing only name and salary; employees whose job is PRESIDENT or ANALYST or SALESMAN; and a case-insensitive search for CLERK.
4. Update an employee's salary using $set and then delete all employees of department 20. What guarantees do you have if the shell crashes in the middle?
5. Describe how you would migrate the dept and emp tables of MySQL into one MongoDB collection that holds each employee with its department name.

16.7 Final revision checklist

- [ ] I can explain why requirement analysis and UI design come before normalization.
- [ ] I can list the steps of getting ready for normalization and recognise the unnormalized form.
- [ ] I can explain the three anomalies with examples.
- [ ] I can normalize a transaction to 1NF, 2NF and 3NF and identify keys and relationships.
- [ ] I can explain BCNF with the city and pin code example and why it is optional.
- [ ] I can explain why normalization can slow queries and what denormalization does.
- [ ] I can describe the MySQL architecture and the role of the storage engine.
- [ ] I can compare InnoDB and MyISAM and name other engines.
- [ ] I can explain temporary tables and how they differ from views.
- [ ] I can recall Codd's foundation rule and the 12 rules with MySQL examples.
- [ ] I can write ON, USING and NATURAL JOIN versions of a join.
- [ ] I can explain NoSQL features, BASE, and vertical vs horizontal scaling.
- [ ] I can explain the CAP theorem and place systems on it.
- [ ] I can state when to use and not use NoSQL and name the four types with examples.
- [ ] I can explain JSON vs BSON, MongoDB installation, mongod and mongo.
- [ ] I can do insert, insertMany, find with criteria and projection, sort/skip/limit, delete and update in MongoDB.
- [ ] I know ObjectId, the 16 MB limit, validators, indexes, $lookup and GridFS.